WEEK 06 · MODEL & VALIDATION

收到資料
不等於可以接受

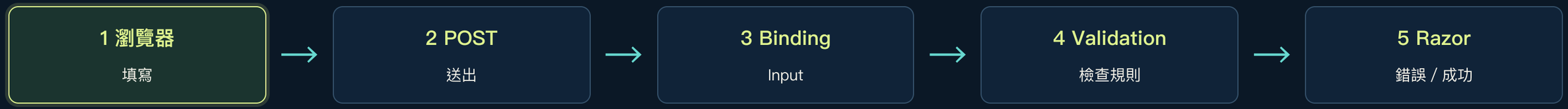
第五週已讓表單資料進入 PageModel；這週把 Input 看懂，再替資料加入清楚規則與可重現測試。

下載學生任務包 ↓　下載 PDF ↓



RECONNECT · 先找回資料路線

驗證不是另一套魔法，它插在收到與成功之間。



下一站　從頭看

先確認第五週的表單仍能把資料送到 Input。

MISSION MAP

三節課，把 Model 從名詞變成可操作的資料結構。

1

看懂 Input Model

從一個屬性開始，理解型別、物件與欄位對應。

2

加入驗證閘門

DataAnnotations 定義規則，ModelState 決定能否成功。

3

用測試證明

走空白、錯誤格式、合法資料三條路，再做同儕驗收。

MODEL 01 · DATA SHAPE

Model 先不用想得很抽象：它描述這筆資料有哪些欄位。

不是畫面

HTML 決定使用者看到與操作什麼。

不是資料庫

物件現在只存在記憶體，重新啟動不會留下。

是一個結構

屬性名稱與型別，說明這筆輸入長什麼樣子。

本週的 Model 指 `RegistrationInput`；PageModel 指整個 Razor Page 的後端處理類別。

MODEL 02 · ONE PROPERTY

一個屬性，就是物件裡一個有名字的位置。

先只看顯示名稱，不一次吞下整個 class。

```
public class RegistrationInput
{{
  public string DisplayName {{ get; set; }} = string.Empty;
}}
```



先讀懂一行，再組成一個物件

MODEL 03 · TYPE

型別先回答：這個位置可以放什麼？

`string`

文字：名稱、Email、場次、備註。

`int`

整數：未來若加入報名人數，可用數值規則。

`bool`

是 / 否：例如是否同意條款；不適合拿來存名稱。

型別不是驗證全部。`string` 能裝下 `abc`，但那不代表它是合法 Email。

MODEL 04 · INPUT MODEL

四個欄位，不要散成四條線；用一個 Input 物件收在一起。

```
public class RegistrationInput
{
    public string DisplayName { get; set; } = string.Empty;
    public string Email { get; set; } = string.Empty;
    public string Session { get; set; } = string.Empty;
    public string? Note { get; set; }
}
```

必填文字

string＋空字串起點

選填文字

string? 可沒有值

物件

四個屬性收成一組

？表示允許沒有值；它不會自動限制長度或 Email 格式。

MODEL 05 · MAPPING

asp-for 寫的路徑，就是 Binding 要找的目的地。

Razor 欄位

```
<input asp-for="Input.DisplayName">
```

C# 目的地

```
public RegistrationInput Input {{ get; set; }}
public string DisplayName {{ get; set; }}
```

> Input.DisplayName 先找 Input，再找裡面的 DisplayName；任何一段拼錯都會斷線。

讓學生用手指出路徑的兩段

暫停講解 · 學生實作

PAUSE & PRACTICE 00

先讀懂 Model，再加規則。

15 分鐘

查看預期成品 Demo ↗

學生現在完成

1. 在紙上或註解中畫出 RegistrationInput 樹狀圖。
2. 標出四個屬性的型別與目前值。
3. 從一個 asp-for 指回正確屬性。

完成條件

- 能區分 PageModel 與 Input Model。
- 能說出 string 只代表文字型別。
- 能指出 Input.DisplayName 的兩段路徑。

老師巡視：先問「這是畫面、PageModel，還是 Input Model？」

第 8 分鐘：只完成 DisplayName 與 Email 也可。

快學生：解釋為何 Model 不是資料庫。

☐ 我已在自己的 localhost 實際完成並確認結果。

時間到：抽一位學生沿 asp-for 指到 C# 屬性

CHECKPOINT 01

能回答三句話，才開始加驗證。

1

PageModel

處理這個 Razor Page 的請求與狀態。

2

Input Model

把這次表單輸入收成一個有結構的物件。

3

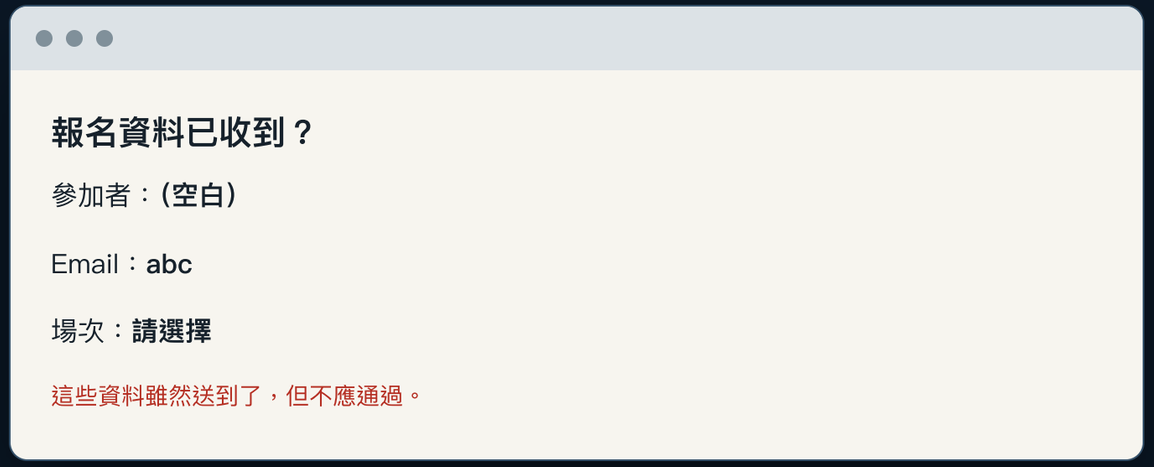
Binding

依欄位名稱，把文字放進對應屬性。

VALIDATION 01 · RECEIVED ≠ VALID

後端收到資料，
不代表資料合理。

空白名字、錯誤 Email、未選場次都能被傳送。驗證的工作，是在成功之前設一道明確閘門。



VALIDATION 02 · RULES BESIDE DATA

每個驗證標記，只回答一個問題。

```
[Required(ErrorMessage = "請輸入顯示名稱")]
[StringLength(20, ErrorMessage = "最多 20 字")]
[Display(Name = "顯示名稱")]
public string DisplayName { get; set; }
```

Required

可以空白嗎？

StringLength

最多可以多長？

Display

畫面上要叫什麼名字？

一條規則，一個可測試問題

VALIDATION 03 · ONE RULE

不要背整串；
逐條問它在防什麼。

[Required]

有沒有填？空白不能通過。

[EmailAddress]

有填時，格式像不像 Email？

[StringLength(20)]

文字長度是否超過上限？

[Display] 改善顯示名稱，本身不是「擋資料」的驗證規則。

一條規則配一條可重現測試

VALIDATION 04 · VALIDATION GATE

先問 ModelState.IsValid，通過才打開成功區。

```
public void OnPost()
{
    if (!ModelState.IsValid)
    {
        return;
    }

    Submitted = true;
}
```

規則未通過

留在表單，顯示錯誤

全部通過

顯示成功摘要

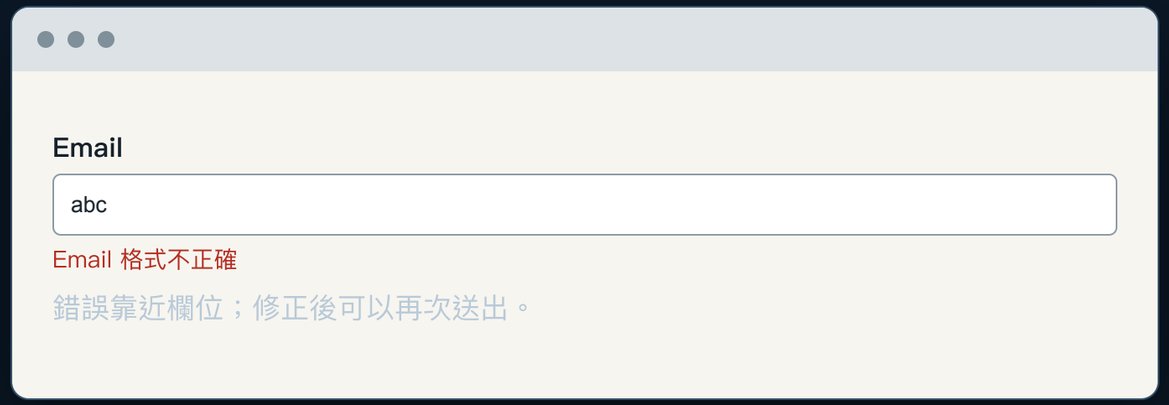
如果先設定 Submitted，再檢查 ModelState，空白資料也可能出現成功訊息。

成功訊息必須在驗證之後

VALIDATION 05 · SHOW THE REPAIR PATH

驗證不是只說「失敗」，要讓使用者知道哪裡需要修。

```
<label asp-for="Input.Email"></label>
<input asp-for="Input.Email"
       class="form-control">
<span asp-validation-for="Input.Email"
      class="text-danger"></span>
```



錯誤訊息本身也是介面設計

暫停講解 · 學生實作

PAUSE & PRACTICE 01

讓錯的停下，讓對的通過。

20 分鐘

查看預期成品 Demo ↗

學生現在完成

1. 名稱加入 Required 與 20 字限制。
2. Email 加入 Required 與 EmailAddress。
3. 場次設為必選。
4. 先檢查 ModelState，再顯示成功。

完成條件

- 空白：錯誤，沒有成功摘要。
- Email=abc：格式錯誤。
- 合法假資料：摘要完全一致。
- 修正錯誤後能再次送出。

老師巡視：要求學生先重現失敗，再改程式。

第 10 分鐘：尚未完成者先做好名稱與 Email 兩項驗證。

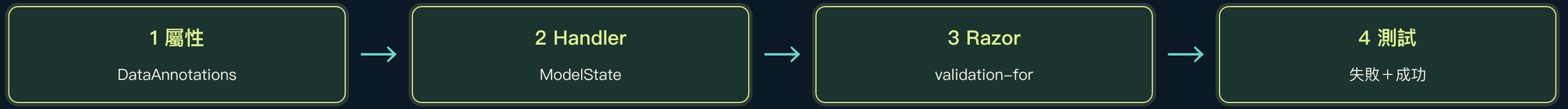
快學生：測試名稱第 20 與第 21 字，不增加欄位。

- [ ] 我已在 localhost 完成空白、錯誤格式、正確資料三條測試。

時間到：兩人互換測試，不直接替對方改程式

CHECKPOINT 02

驗證有四個合作位置，少一個就可能看不到結果。



> 先說明哪一站不符合預期，再做最小修改。

TEST 01 · TEST BEHAVIOR

只測成功一次，不能證明驗證有效。

測試路線	輸入	應該看到	操作
空白	所有必填欄空白	必填錯誤；沒有成功摘要	模擬
錯誤格式	Email = abc	Email 格式錯誤	模擬
正確	課堂假資料	成功摘要與輸入一致	模擬

選一條路，觀察預期行為。之後要在自己的 localhost 重做。

TEST 02 · BOUNDARY

上限 20 字，至少要測 20 字成功、21 字失敗。

19

上限內

應成功，但還沒碰到真正邊界。

20

剛好上限

應成功，證明沒有少允許一個字。

21

超過一個

應失敗，證明後端規則真的生效。

TEST 03 · AI DEBUG CONTRACT

不要請 AI 重寫整頁，給它一個可重現的小問題。

✕ 太模糊

```
表單壞了，幫我全部修好。
```

AI 不知道輸入、操作、預期與實際結果。

✓ 可驗證

```
Razor Pages 表單使用 POST。
輸入 Email=abc 時仍顯示成功。
以下只貼 OnPost 與 Email 屬性。
請先指出原因，再給最小修改與三組測試。
```

讀 理解建議 → **比** 查看差異 → **跑** 啟動專案 → **測** 三條路 → **追問** 描述現象

暫停講解 · 學生實作

PAUSE & PRACTICE 02

用失敗與成功證明規則真的存在。

20 分鐘

查看預期成品 Demo ↗

學生現在完成

1. 送出所有必填欄空白。
2. 輸入 Email=abc。
3. 使用合法假資料。
4. 加測名稱 20 字與 21 字。

完成條件

- 失敗時不顯示成功摘要。
- 錯誤訊息靠近對應欄位。
- 合法摘要與輸入完全一致。
- 能重現並說明一個邊界值。

老師巡視：要求學生先說預期，再按送出。

第 10 分鐘：至少完成空白、abc、合法三路。

快學生：測 20 / 21 字並說明邊界。

☐ 我已在自己的 localhost 實際完成並確認結果。

時間到：留下三條可重現的測試證據

PAIR TEST

測試者不改程式，只交付一份可重現的回報。

1

輸入

清楚說明用了哪一組假資料。

2

操作

說明按了什麼、在哪個畫面發生。

3

結果

比較預期與實際，不只說「壞掉」。

回報句型：「我輸入____，按下送出後，預期____，實際看到____；請先檢查____。」

使用課堂假資料；不要輸入真實 Email、電話、密碼、連線字串或 API key。

FINAL BOSS · REPAIR WITH EVIDENCE

不是改一個字就過關，要重現、修正、測試、說明。

基礎救援
一個問題，有明確修改範圍

標準任務
自己定位，測成功與失敗

進階委託
加入小需求與邊界測試

抽情境任務

基礎救援　8–12 分鐘

幽靈報名成功

情境：所有欄位空白，畫面卻顯示報名成功。

任務：調整 OnPost，讓錯誤資料停在表單。

限制：保留 DataAnnotations；必須使用 ModelState.IsValid。

必測：空白失敗；合法資料成功。

口頭說明：為什麼要先檢查再設定 Submitted？

☐ 老師已看過我的重現、修改、測試與說明，確認通過。

同級抽題，避免運氣造成難度失衡

MISSION COMPLETE · WEEK 06

你現在不只會接資料，還能解釋、擋錯、測試。

Model 說得清楚

分得出 PageModel、Input Model 與資料庫。

規則有閘門

DataAnnotations 與 ModelState 各負責一件事。

測試有證據

空白、錯誤格式、合法資料都能重現。

☐ 我能解釋 Input Model ☐ 驗證規則與閘門完成 ☐ 三路測試有證據 ☐ 老師已驗收 Boss

仍未寫入資料庫；下一步才是多筆清單與持久保存。